Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency

Author names to be supplied for arXiv v1

July 2026

Abstract

Joint-embedding predictive architectures (JEPAs) learn world models that predict in a compact latent space rather than in pixels, reducing the pressure to model nuisance appearance. Yet this provides no guarantee against visual perturbations: they can still alter the encoded representation and affect subsequent action-conditioned predictions. Bisimulation captures this requirement precisely: two observations should be treated as the same state only when their action-conditioned consequences agree. Guided by this criterion, we introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic that measures how far a clean history and a visually perturbed view of it diverge after being rolled forward under the same action sequence. We prove that this divergence bounds the perturbation-induced change in multi-step prediction error and planner cost. Two complementary measures complete the diagnostic: the Invariance Radius quantifies how far a clean view and its perturbed counterpart spread after prediction, and the Separation Rate checks that different states remain distinguishable. Experiments on four visual control tasks show that pairwise ACPC predicts perturbation-induced prediction and cost changes. On LeWM, the ACPC-based screen transfers to new tasks and visual shifts, while PLDM exhibits similar diagnostic trends under a different architecture. Code is available here.

Keywords: visual world models; predictive consistency; action-conditioned rollouts; visual invariance; checkpoint diagnostics.

1 Introduction

World models support control by predicting the outcomes of candidate actions [1, 2]. Joint-embedding predictive architectures (JEPAs) predict target representations rather than reconstructing pixels, avoiding an explicit requirement to reproduce nuisance visual details [3, 4, 5, 6, 7]. Such representations should be insensitive to task-irrelevant visual perturbations while preserving distinctions between situations that require different actions. This requirement echoes the intuition behind bisimulation, which defines state equivalence through action-conditioned consequences rather than visual appearance [8, 9]. However, encoder distances alone do not show how a perturbation propagates through prediction. Over a multi-step rollout, the predictor may amplify or contract the initial representation difference. Encoder-level and single-transition diagnostics do not directly measure this rollout-level effect. We therefore ask: when a clean history and its perturbed view are rolled forward under the same action sequence, how far apart are their predicted trajectories?

To measure this rollout-level effect, we introduce Action-Conditioned Predictive Consistency (ACPC). ACPC rolls a clean history and its perturbed view forward under the same action sequence and measures the distance between their predicted trajectories. Figure 1 provides a conceptual overview of how this pairwise distance is summarized by IR and complemented by SR. Low ACPC alone, however, does not rule out representational collapse: a constant representation would make every paired rollout identical. We therefore summarize ACPC across histories with the *Invariance Radius* (IR), where lower values indicate less sensitivity to visual perturbations. The *Separation Rate* (SR) checks whether different states remain distinguishable after rollout, with higher values indicating better separation.

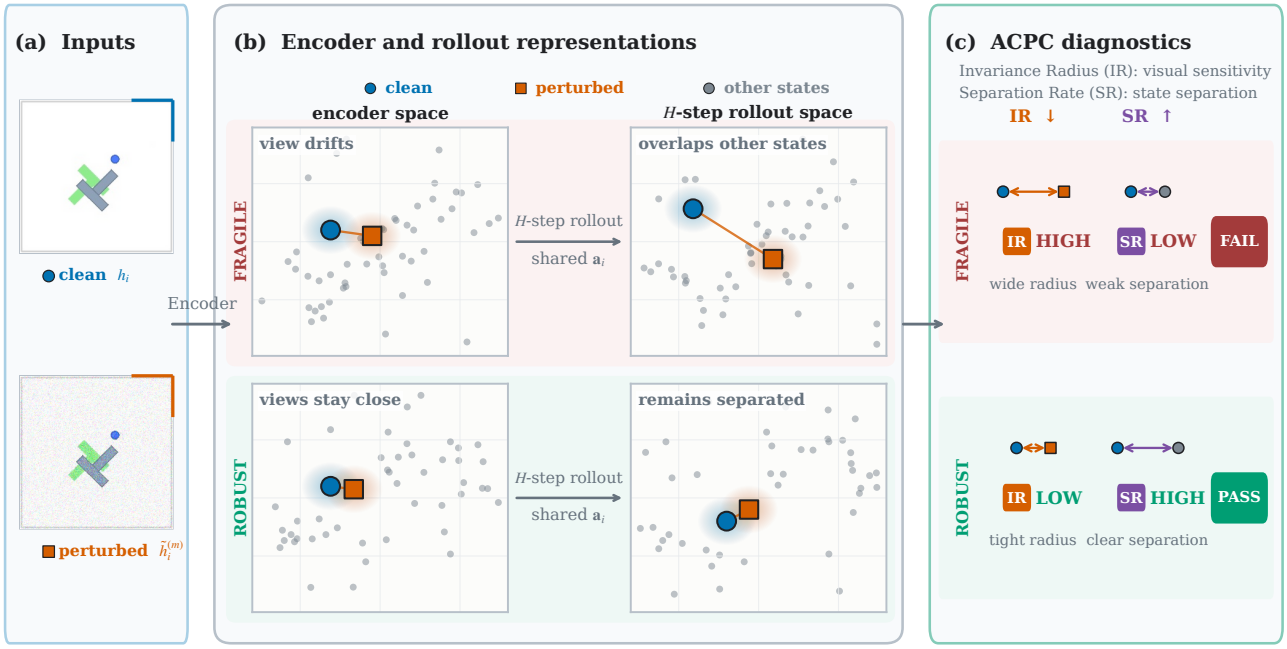

Figure 1: Conceptual overview of ACPC, IR, and SR. (a) A logged clean history and a visually perturbed view of it form the paired inputs. **(b)** A frozen world model encodes both views and rolls them forward for H steps under the same recorded action sequence. ACPC measures the distance between the two predicted trajectories. In the fragile case, perturbation-induced spread grows and overlaps representations of other states (gray); in the robust case, the paired views remain close while other states stay separated. Layouts are schematic: only within-panel proximity is meaningful, and positions enter no reported metric. **(c)** The Invariance Radius (IR) summarizes perturbation-induced rollout spread across histories, while the Separation Rate (SR) measures how often selected different-state rollouts remain separated beyond this spread. A checkpoint passes the diagnostic screen only if IR is low and SR is high; passing is specific to the evaluated visual shift and state labels and does not certify robustness.

We prove that ACPC bounds the perturbation-induced change in multi-step prediction error. When the same candidate action sequences are evaluated from both views, ACPC also yields bounds on changes in their predicted costs (Propositions 1 and 2). These bounds hold for each evaluated pair and require no distributional or smoothness assumptions. Experiments on LeWM [10] show that ACPC is informative about prediction-error change and selection regret for the cross-entropy method (CEM) [11]. Together, IR and SR help identify checkpoints that perform well under visual perturbations across tasks and perturbation types. A PLDM [12] sweep examines how the same diagnostic quantities behave under a different world-model architecture.

The contributions are:

1. We introduce **Action-Conditioned Predictive Consistency (ACPC)**, which compares the predicted trajectories of a clean history and its perturbed view under the same action sequence. IR summarizes sensitivity across histories, while SR checks whether different states remain distinguishable (Section 3).
2. We prove **samplewise bounds** on how much a visual perturbation can change multi-step prediction error. When the same candidate action sequences are evaluated from both views, we also bound changes in their predicted costs (Propositions 1 and 2).
3. We evaluate **ACPC, IR, and SR** across four control tasks and three visual perturbations, and examine their behavior on a second world-model architecture. ACPC provides predictive information about perturbation-induced changes in multi-step prediction error and planning cost, while IR and SR help identify trained models that maintain control performance (Section 4).

2 Related Work

World Models. World models predict how an environment evolves under actions and use those predictions for control. DreamerV3 [1] learns from imagined trajectories, while TD-MPC2 [2] plans with learned latent dynamics. Joint-embedding predictive architectures (JEPAs) learn representations by predicting targets in representation space rather than reconstructing pixels [3, 4, 5, 13]. LeWM [10], our primary model family,

combines this joint-embedding objective with action-conditioned latent prediction. PLDM [12] provides a different joint-embedding dynamics architecture on which we also evaluate ACPC. Earlier work analyzes how joint-embedding predictive objectives can favor slow features [14]. Related training objectives predict future latent observations from histories and actions in PBL [15], or match predicted and target representations under augmentation in SPR [16]. Subsequent work studies collapse in self-predictive learning [17] and the role of action conditioning [18]. Theoretical analyses ask when latent prediction can disregard irrelevant features or favor predictable, high-influence features [6, 7] and when it can recover latent state up to a linear transformation [19]. seq-JEPA [20] further studies invariance and equivariance under known view transformations. These works explain how predictive representations and dynamics are learned; we evaluate how a trained predictor responds when its visual input is perturbed.

Robustness to Visual Perturbations. Training-time augmentation improves visual control in DrQ [21] and DrQ-v2 [22], while SODA [23] uses augmentation in an auxiliary representation-learning objective. ViGMO [24] and VIBR [25] learn invariance in latent or value space. ReOI [26] instead modifies observations at test time to reduce distractor sensitivity in visual model-predictive control. Other methods change world-model training. TPC [27] favors temporally predictable information, and DreamerPro [28] replaces pixel reconstruction with prototype prediction. Task Informed Abstractions [29] and Denoised MDPs [30] use task or reward information to separate useful state from distractors. Iso-Dream [31] separates controllable and noncontrollable dynamics, while other approaches combine temporal masking with bisimulation [32], infer the actions of visually similar distractors [33], or use task-aware reconstruction [34]. These methods reflect a broader principle: robustness should remove irrelevant visual variation without discarding distinctions needed for control. Bisimulation formalizes behavioral similarity through rewards and action-conditioned transitions [8, 9, 35], including a reward-free formulation based only on transitions [36]. Value-equivalent models and value-aware model-learning objectives instead focus on information needed for value prediction or planning [37, 38]. Most closely related, MWM [39] enforces action-conditioned rollout consistency during training. ACPC measures how clean and perturbed latent rollouts diverge in a frozen model under the same action sequence, without retraining it.

Diagnostics for World Models. Model accuracy and control performance do not always move together: one-step likelihood can be a poor guide to downstream performance [40]. This mismatch has motivated methods that connect model learning and evaluation to downstream decisions [41]. World-model evaluation should therefore reflect the intended use of the model, including long rollouts, planning, and policy evaluation [42]. One group of methods tests or enforces whether learned dynamics respect actions. ATM [43] compares action information in encoded and predicted transitions, while Delta-JEPA [44] decodes actions from latent differences. ACID [45] adds inverse-dynamics cycle consistency to the planning cost. Future-Compatible [46] checks whether generated futures agree with their stated actions, and World Models as Group Actions [47] tests identity, inverse, and composition structure. Related diagnostics expose kinematic failures in long rollouts [48] or use frozen inverse-dynamics probes to test whether latents retain action information under visual corruptions [49]. A second group evaluates downstream failure. WMAttack [50] searches for visual attacks on closed-loop world-model agents, while ARB4WM [51] targets their policy, value, and latent dynamics. Other work compares predicted multi-step latent transitions with those produced by the environment [52], or compares predicted and true plan costs [53]. Finally, CARRL [54] certifies Q-based action choices under bounded observation uncertainty, and CROP [55] certifies per-state actions and finite-horizon reward bounds through smoothing. These certificates cover policy decisions or return, whereas our bounds preserve only the winner or elite set for an evaluated pair and fixed candidate pool. More broadly, existing diagnostics assess action content, rollout validity, model error, or attack sensitivity. ACPC asks how a task-preserving visual perturbation propagates through a shared-action rollout without requiring the true future.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A visual perturbation can change the trajectory predicted by a world model. ACPC measures this change by rolling a clean history and its perturbed version forward under the same action sequence and comparing the two trajectories. We first define this pairwise measurement. We then show that it bounds perturbation-induced changes in prediction error and candidate cost, which leads to conditions for preserving a planner's selected candidate. To summarize ACPC across a checkpoint, IR captures clean–perturbed pairs with high sensitivity, while SR checks that different states remain separated and prevents a collapsed representation from appearing robust. Finally, we combine IR and SR into an empirical checkpoint screen.

3.1 Pairwise ACPC

Given a clean history and its perturbed version, ACPC compares the rollouts predicted from them under identical actions. Let h denote the clean history and let $\tilde{h}$ be the result of applying the evaluated visual perturbation to h. The frozen encoder maps them to $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both representations are then rolled forward under the action sequence $\mathbf{a} = (a_0, \ldots, a_{H-1})$. Let F_θ denote the frozen action-conditioned predictor. After the first k actions, the two predicted representations are

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \qquad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \tag{1}$$

Here H is the number of autoregressive future steps. ACPC compares predictions in the projected latent space used to evaluate planner costs. Let Π map each predicted representation into this space. To combine all H predicted steps into one trajectory-level distance, we assign nonnegative weights α_k, with $\sum_{k=1}^{H} \alpha_k = 1$, and define the weighted rollout vector

$$\bar{G}_{\mathbf{a}}(z) = \left[\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \cdots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top\right]^\top. \tag{2}$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\|\bar{G}_{\mathbf{a}}(E_\theta(h)) - \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))\right\|_2 = \left(\sum_{k=1}^{H} \alpha_k \left\|\Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k)\right\|_2^2\right)^{1/2}. \tag{3}$$

Low ACPC means that the two predicted trajectories remain close; high ACPC means that they separate. Encoder shift $\|z - \tilde{z}\|_2$ measures the difference before prediction, whereas ACPC measures it after rollout. Using the same action sequence removes action differences from the comparison. ACPC measures rollout consistency, not prediction accuracy.

We use uniform weights $\alpha_k = 1/H$ and $H = 8$, the longest horizon available for every logged window (Appendix C). The planner analysis uses $H = 5$ to match the CEM planning horizon (Section 4.5).

3.2 Prediction-Error Bounds

ACPC itself does not require the observed future. To connect it to prediction error, let $Y_{\mathbf{a}}^H$ denote the observed future under the same action sequence, represented in the same weighted space as the predicted rollouts (Equation (2)). Both rollout errors are measured against this single target. Their difference cannot exceed the distance between the two predictions, which is ACPC.

Proposition 1 (Prediction-error change bound)**.** *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_\theta(h)) - Y_{\mathbf{a}}^H\|_2, \qquad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) - Y_{\mathbf{a}}^H\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{4}$$

In particular, the increase in error caused by the perturbation, $(e_{\tilde{h}} - e_h)_+$, is also bounded by ACPC.

This result follows directly from the reverse triangle inequality; the proof is given in Appendix A. The bound concerns the change in error, not absolute prediction accuracy. Both rollouts can be inaccurate even when ACPC is small. In the experiment, the observed future is used to compute the error change $d = |e_{\tilde{h}} - e_h|$, not ACPC itself (Section 4.4).

3.3 Selection Stability from Planning-Cost Bounds

A planner ranks candidate action sequences by their predicted costs. Changing the input can change each candidate's predicted endpoint and therefore its cost. If these cost changes are too small to close the gaps between candidates, the selected candidate remains unchanged. We now formalize this idea.

For each candidate action sequence $\mathbf{a}^j$, we roll both inputs forward under that sequence. Let

$$x_j = \Pi(F_\theta^H(E_\theta(h), \mathbf{a}^j)), \qquad \tilde{x}_j = \Pi(F_\theta^H(E_\theta(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is one component of the candidate-specific ACPC. Whenever $\alpha_H > 0$,

$$r_j \leq \frac{\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}}.$$

Proposition 2 (Planning-cost bounds)**.** *Suppose the planner scores candidate j by its squared distance to a fixed goal embedding g:*

$$C_j = \|x_j - g\|_2^2, \qquad \tilde{C}_j = \|\tilde{x}_j - g\|_2^2.$$

Define the candidate-specific cost-change bound

$$b_j = r_j\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big). \tag{5}$$

Then $|\tilde{C}_j - C_j| \leq b_j$.

Let w and $\tilde{w}$ be the winners under the clean and perturbed inputs. If the winner changes, the perturbed winner's excess cost under the clean prediction satisfies

$$0 \leq C_{\tilde{w}} - C_w \leq b_{\tilde{w}} + b_w. \tag{6}$$

The bound b_j is computed from the evaluated endpoints and requires no global Lipschitz constant. If the clean winner w leads every competitor j by more than $b_w + b_j$, the perturbation cannot reverse their order. The same argument preserves the top-k elite set when every clean elite–non-elite gap exceeds the corresponding pair of bounds. Appendix A gives the exact conditions (Proposition 3).

Because r_j is bounded by candidate-specific ACPC, substituting its bound into b_j gives

$$|\tilde{C}_j - C_j| \;\leq\; b_j \;\leq\; \frac{\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}}\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big), \tag{7}$$

Thus, ACPC bounds how much each candidate's cost can change, while the clean cost gaps determine whether that change can alter the planner's selection. These certificates require evaluating every candidate under both inputs, so they support post-hoc analysis rather than online screening.

CEM plans in rounds. In each round, it scores a set of candidate action sequences, keeps the best group (the elite set), and uses that group to generate the candidates for the next round [11]. We compare a clean and perturbed run that start from the same proposal and use the same random samples. If the perturbation does not change the elite set in any round, both runs generate the same candidates in the next round and eventually return the same action (Corollary 1). If the elite set changes, the guarantee no longer applies. This result concerns the planner's model-based choice, not its return in the environment.

3.4 Checkpoint-Level IR and SR

Pairwise ACPC measures one clean–perturbed pair under one action sequence. Evaluating a checkpoint requires summarizing this measurement across many histories. The *Invariance Radius* (IR) measures how sensitive these paired rollouts are to the visual perturbation; lower IR is better. Low IR alone is not enough because a collapsed representation would make every rollout look similar. The *Separation Rate* (SR) therefore checks whether different states remain distinguishable after rollout; higher SR is better. A favorable checkpoint should have both low IR and high SR.

To compute IR, we select n logged history windows as anchors. Anchor i provides a clean history h_i and its recorded action sequence $\mathbf{a}_i = (a_{i,0}, \ldots, a_{i,H-1})$. We apply the visual perturbation M times to obtain $\tilde{h}_i^{(1)}, \ldots, \tilde{h}_i^{(M)}$, and compute ACPC for each clean–perturbed pair under the same recorded actions.

Raw latent distances can have different scales across histories. We therefore divide each ACPC value by the anchor's typical one-step clean motion, denoted by s_i. Specifically, s_i is the median projected displacement between consecutive clean observations in the anchor window (Equation (23) and appendix C). This expresses ACPC relative to the amount of motion normally seen in that history. With a small numerical stabilizer $\varepsilon_{\text{norm}}$, define

$$R_i^{(m)} = \frac{\text{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\text{norm}}}, \qquad \bar{R}_i = \frac{1}{M}\sum_{m=1}^{M} R_i^{(m)}. \tag{8}$$

The average $\bar{R}_i$ gives one normalized sensitivity value for each anchor. Let Q_q denote the empirical q-quantile across anchors. The raw IR is

$$\text{IR}_q^{\text{raw}}(\theta) = Q_q\big(\{\bar{R}_i\}_{i=1}^n\big). \tag{9}$$

IR is one value for the checkpoint. We use $q = 0.90$ so that IR reflects anchors with relatively high sensitivity, which a mean could hide. The result is stable for q between 0.80 and 0.95 and for the tested rollout horizons (Table 1).

IR checks whether perturbed views remain close to their clean counterparts. SR checks whether this invariance also preserves differences between states. For each eligible anchor, the protocol selects a nearby logged history with a different state-coordinate label. Both histories are rolled forward under the anchor's recorded actions, so their separation is not caused by different action sequences. Their trajectory distance is normalized by the same clean-motion scale used for IR and is denoted by D_i^{diff} (Equation (24) and appendix C).

Let $\mathcal{I}$ be the set of anchors for which such a comparison is available. SR is the fraction whose different-state distance exceeds raw IR by a margin δ:

$$\text{SR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{1}\big[D_i^{\text{diff}} > \text{IR}_q^{\text{raw}}(\theta) + \delta\big], \tag{10}$$

We use $q = 0.90$ and $\delta = 0.10$. SR therefore reports how often a tested different-label pair remains beyond the checkpoint's perturbation radius plus the fixed margin.

A collapsed representation makes IR small, but it also makes every D_i^{diff} zero and therefore fails SR. Conversely, a checkpoint can have high SR while remaining sensitive to visual perturbations, so IR is still needed. SR applies only to the state-coordinate labels used in the tested pairs. Appendix C gives the exact pairing rules and eligible-pair counts.

3.5 Checkpoint Screening

SR uses raw IR because it compares distances within the same checkpoint. To compare a trained checkpoint with its unaugmented reference, we use relative IR. Let θ_0 denote the unaugmented checkpoint from the same task, training run, and model family. We define

$$\text{IR}_q^{\text{rel}}(\theta;\theta_0) = \frac{\text{IR}_q^{\text{raw}}(\theta)}{\text{IR}_q^{\text{raw}}(\theta_0)}. \tag{11}$$

Relative IR equals 1 for the reference checkpoint. Values below 1 indicate lower sensitivity than the reference, while values above 1 indicate higher sensitivity. This comparison is defined within the same task, training run, and model family; it does not make IR directly comparable across them.

A checkpoint passes the screen only when relative IR is below its threshold and SR is above its threshold. We combine these two requirements into one score. Each term measures the checkpoint's normalized margin from one threshold, and the minimum selects the weaker of the two results.

For model family $\mathcal{F}$ and visual shift υ, let the thresholds be t_{IR} and t_{SR}. All measurements in a score use the same shift. With a small numerical constant $\varepsilon_{\text{score}}$, define

$$S_{\mathcal{F},\upsilon}(\theta;\theta_0) = \min\left\{\frac{t_{\text{IR}} - \text{IR}_q^{\text{rel}}(\theta;\theta_0)}{|t_{\text{IR}}| + \varepsilon_{\text{score}}}, \frac{\text{SR}_{q,\delta}(\theta) - t_{\text{SR}}}{|t_{\text{SR}}| + \varepsilon_{\text{score}}}\right\}. \tag{12}$$

The score is nonnegative exactly when both requirements pass. A negative score means that at least one requirement fails.

We choose the two thresholds using planning success on source tasks and apply them unchanged to held-out tasks. Thresholds are selected separately for each model family. Once fixed, the score requires no task-success labels.

To compare checkpoint θ_1 with its reference θ_0, we report the change in screening score: $\Delta S_{\mathcal{F},v}(\theta_1;\theta_0) = S_{\mathcal{F},v}(\theta_1;\theta_0) - S_{\mathcal{F},v}(\theta_0;\theta_0)$. A positive ΔS means that θ_1 receives a better joint IR–SR score than the reference. It does not label either checkpoint as robust.

The experiments evaluate the method at two levels. Pair-level experiments test whether ACPC reflects changes in prediction error and planner selection (Sections 4.4 and 4.5). Checkpoint-level experiments test whether a fixed IR–SR screen transfers across LeWM tasks and visual shifts, and separately examine how IR and SR behave on PLDM (Sections 4.6 to 4.8). The checkpoint result is an empirical screening claim; it does not follow from the pairwise bounds.

4 Experiments

After describing the evaluation protocol, we evaluate the diagnostic at two levels. At the pair level, we test whether ACPC reflects prediction-error change and CEM selection regret. At the checkpoint level, we examine how IR and SR vary across LeWM recovery, whether checkpoint screening transfers across tasks and visual shifts, and whether similar diagnostic trends appear on PLDM. We begin with a local-geometry case study to visualize the representation pattern behind these measurements.

4.1 Evaluation Protocol

Models, Tasks, and Evaluation. Most experiments use LeWM [10]. We also evaluate PLDM [12] to test the diagnostic on a second world-model architecture. We use four control tasks: TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. For planning evaluation, CEM uses the frozen world model to optimize five-step action sequences. We report the *planning success rate*, defined as the percentage of episodes that reach the task goal.

We use *visual perturbation* for one transformed history and *visual shift* for the distribution of such transformations. The main experiments add Gaussian noise with $\sigma = 0.08$ to the history observations while keeping the goal image clean. Additional experiments use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each task and model family, the Gaussian-noise sweep contains nine training conditions: one without noise augmentation and eight with full-sequence Gaussian-noise augmentation at $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$. Both LeWM and PLDM have three independent training runs for each task–condition pair (seeds 3072/3073/3074). Each trained checkpoint is evaluated with seeds 42/43/44, using 100 episodes per evaluation seed. We use *task–run cell* for one task and one training run.

The training run is the unit of replication. The three evaluation seeds measure only within-run variability, so the reported means and dispersions across runs are descriptive rather than population estimates.

Diagnostic Protocol. IR uses 100 logged anchors, five Gaussian-noise draws at the evaluation severity $\sigma = 0.08$ (blur and resize diagnostics instead apply the corresponding shift), eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. To compute SR, the protocol selects, for each anchor, a nearby history with a different endpoint-state label and checks whether the two rollouts remain farther apart than raw IR plus 0.10. Appendix C gives the exact pairing, normalization, and threshold rules. Checkpoint comparisons use relative IR from Equation (11).

Threshold Protocol. A checkpoint in the LeWM Gaussian-noise training sweep meets the *success-rate criterion* if it recovers at least 80% of the improvement from the unaugmented checkpoint to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations. Both constants were fixed a priori. We select $(t_{\mathrm{IR}}, t_{\mathrm{SR}})$ on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks. The one-, two-, and three-task selection settings produce $4 + 6 + 4 = 14$ directional threshold-selection/test partitions. Metrics first average training runs within each test task and then weight tasks equally; checkpoint rows are never treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, IR, or SR.

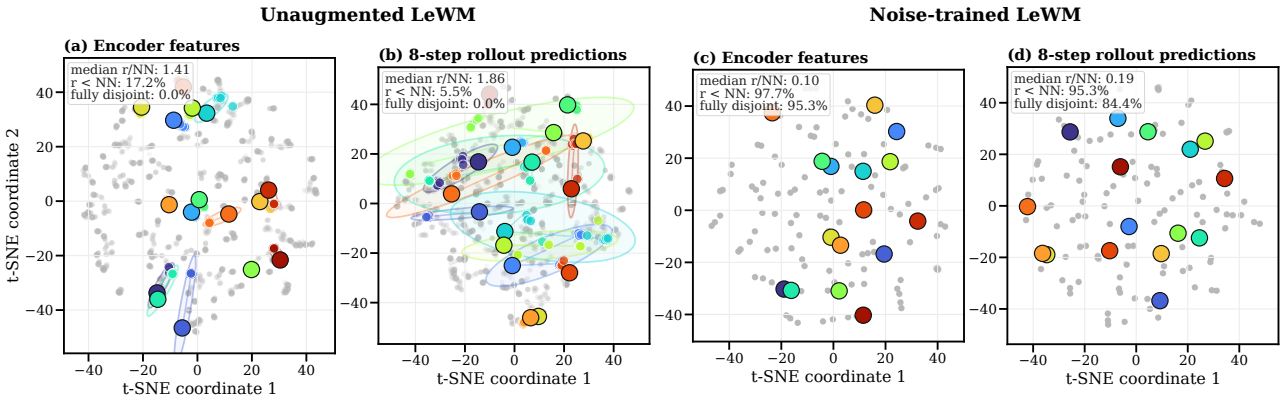

Figure 2: Descriptive local-geometry case study motivating the controlled ACPC comparison. Panels (a)–(d) compare a matched pair of PushT checkpoints—the left two unaugmented, the right two trained with Gaussian-noise augmentation at $\sigma_{\max} = 0.08$—at the encoder output and after eight autoregressive rollout steps. Each color denotes one of 16 highlighted histories: the large black-rimmed marker is its clean anchor, smaller same-color markers are its 18 perturbed views, and the faint ellipse is their 90% t-SNE envelope; gray points show all other histories and views. Under strong contraction, smaller markers and ellipses can lie beneath the clean marker. Each panel's upper-left summary reports, across all 128 anchors, the median r/NN and the percentages with $r < \mathrm{NN}$ and with fully disjoint high-dimensional enclosing balls. The t-SNE coordinates and ellipses are qualitative and enter none of these metrics.

For blur and resize, each task uses the thresholds selected under Gaussian noise on the other three tasks. We compare 24 checkpoint pairs (four tasks $\times$ three runs $\times$ two shifts). Each pair contains the unaugmented checkpoint and the checkpoint trained at $\sigma_{\max} = 0.08$. A pair is positive when success under the shift improves by at least five percentage points and clean success decreases by no more than five points. We make no blur- or resize-specific adjustment. This analysis also does not benchmark alternative signals such as encoder shift or one-step ACPC.

4.2 Local Geometry of Perturbed Views

We begin with a PushT case study showing how visual noise can make perturbed views of one history overlap with other histories. For one clean history and its perturbed views, the sampled visual radius r measures the spread of the perturbation cloud; nearest-clean spacing NN is the distance to the nearest other clean history. The ratio r/NN asks whether visual variation overwhelms local between-history spacing, while a symmetric two-ball test counts fully disjoint clouds. These quantities are descriptive: neighbors are chosen in learned space, each clean rollout follows its own recorded actions, and a collapsed representation makes every radius small. They motivate the controlled ACPC comparison but do not replace it.

Using a fixed PushT case study, we compare a matched pair of LeWM checkpoints: one unaugmented and one trained with full-sequence Gaussian-noise augmentation at $\sigma_{\max} = 0.08$. We inspect both the encoder output and the representation after eight autoregressive rollout steps. All ratios and fractions use the original 192-dimensional projected latent space in which the planner computes costs; formal definitions and sampling details are in Appendix B. The t-SNE visualization [56] is qualitative only.

Figure 2 summarizes the case study. Without augmentation, the median r/NN is 1.41 at the encoder and 1.86 after eight rollout steps, and none of the 128 anchor clouds is fully disjoint. After Gaussian-noise augmentation, the corresponding ratios fall to 0.10 and 0.19; the fully disjoint fractions rise to 95.3% at the encoder and 84.4% after the rollout. Thus, in this fixed case study, augmentation makes perturbed views of the same history substantially more compact relative to nearby clean histories. Much of this separation remains after prediction.

This case study shows that augmentation can contract perturbation-induced variation relative to nearby clean histories, including after prediction. It does not establish task-relevant separation or planning robustness. We therefore turn to checkpoint-level IR and SR and their relationship with planning performance across the complete augmentation sweep (Figure 3).

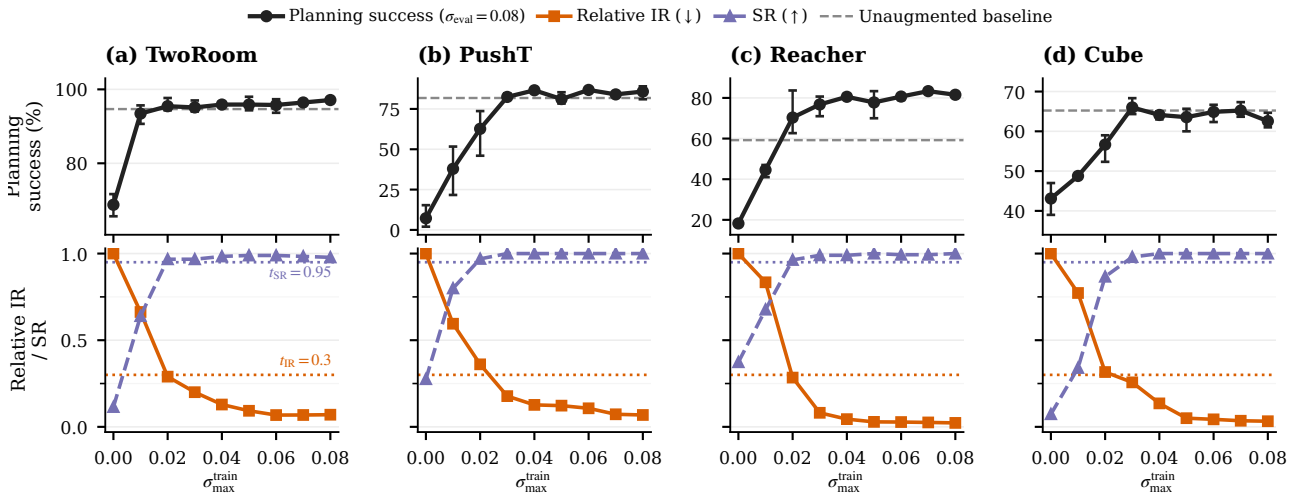

Figure 3: Checkpoint-level planning performance and diagnostics across the complete Gaussian-noise training sweep. We report planning success rate at evaluation noise $\sigma = 0.08$, relative IR, and SR. Points are across-run means; success-rate error bars span the three training runs, and success-rate axes are scaled per task. The dashed line is the clean success rate of the unaugmented checkpoint; the leftmost black point is that checkpoint evaluated at $\sigma = 0.08$. Relative IR equals one there by construction. Lower IR and higher SR are favorable. Dotted lines mark $t_{\text{IR}} = 0.3$ and $t_{\text{SR}} = 0.95$ (Section 4.6).

4.3 IR and SR across Checkpoint Recovery

At evaluation noise $\sigma = 0.08$, the unaugmented LeWM checkpoints lose 25.9 ± 2.4, 74.6 ± 5.6, 41.0 ± 1.4, and 22.1 ± 2.8 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian-noise augmentation recovers performance over a range of training levels whose location differs by task. Figure 3 compares success rate with IR and SR across the complete sweep.

On Reacher, augmentation also raises clean success from 59.2% to 76–82%. Its recovery under noisy evaluation therefore cannot be attributed to robustness alone; the checkpoints also improve on clean observations.

Every augmentation level that meets the success-rate criterion has lower IR and higher SR than its unaugmented reference, although neither measure is monotone at every level. Across the 108 LeWM checkpoint rows, 77 pass the reported IR threshold $t_{\text{IR}} = 0.3$, and all 77 also pass $t_{\text{SR}} = 0.95$. Thus every accepted checkpoint combines reduced same-history sensitivity with retention of the evaluated state-coordinate distinctions. The different-label distance medians are largely stable across augmentation levels, so the rise in SR mainly reflects contraction of the same-history radius rather than expansion of different-label distances.

IR tracks the recovery region, while SR verifies that the sampled state-coordinate distinctions remain outside the contracted radius. A first-order estimate of the composed encoder–rollout Jacobian offers one mechanism for the IR reduction (Appendix G). The next two experiments test whether ACPC uses recorded-action information and whether it tracks a planner-facing quantity (Sections 4.4 and 4.5).

4.4 ACPC and Prediction-Error Change

Does ACPC help predict how much a visual perturbation changes multi-step prediction error? For each history pair, Proposition 1 shows that the two errors against the same logged future can differ by at most ACPC_H. We test whether measured ACPC is informative about this bounded quantity, the error drift $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor; Table 1 shows that the checkpoint-level conclusions are stable for $H \in \{1, 2, 4, 8\}$.

Data and protocol. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian-noise standard deviations $\sigma \in \{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16, and all rows from one trajectory group stay together. We report the mean absolute error (MAE) on groups excluded from model fitting.

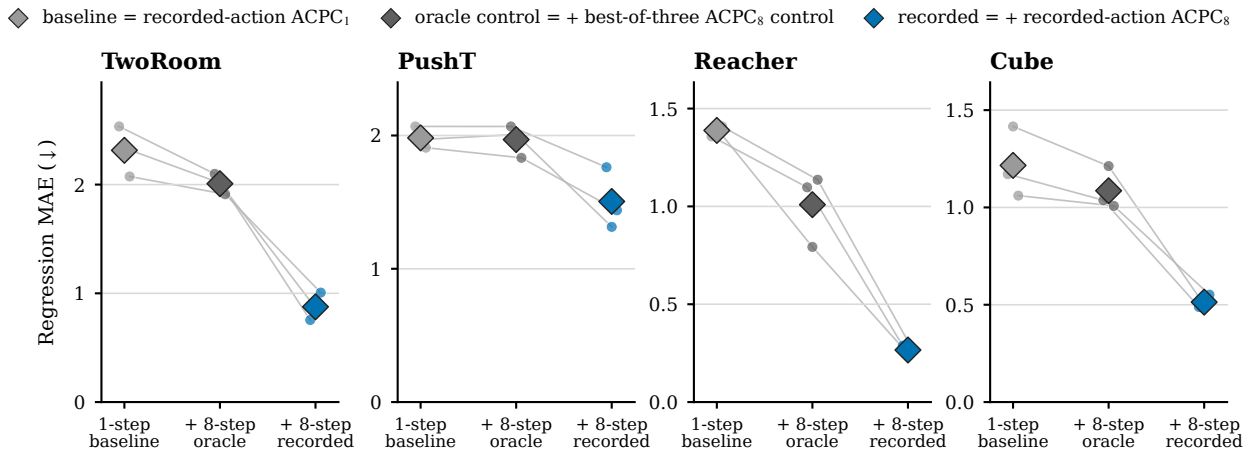

Figure 4: Cross-validated MAE for predicting the error drift d on the unaugmented checkpoints; lower is better. Each model is evaluated on trajectory groups excluded from fitting. Small points are training runs, thin lines join the same run, and diamonds are task means. The in-figure legend labels correspond to Base, Base+Control$_8$, and Base+ACPC$_8$ as defined in Section 4.4. Base+ACPC$_8$ is lowest in all 12 task–run cells; latent-error scales are task-specific, so compare within a panel.

Three nested regressions. Every model contains the probe severity and the encoder-history distance $\|z - \tilde{z}\|_2$.

- **Base** adds one-step ACPC under the recorded action: the information available without a multi-step rollout.
- **Base+Control$_8$** adds the strongest *destroyed-action control*: eight-step ACPC recomputed with *zeroed* actions (every action set to zero), *swapped* actions (the recorded actions of a different trajectory), or *shuffled* actions (the anchor's own actions in permuted temporal order). Each control performs the same eight-step rollout, so it carries rollout length without the recorded action sequence. "Strongest" is an oracle choice: in each task–run cell we report whichever of the three controls attains the lowest test MAE, which makes the comparison conservative.
- **Base+ACPC$_8$** instead adds eight-step ACPC under the recorded actions—the only feature whose action sequence is the one that generated the logged future, and hence the feature matching the right-hand side of Proposition 1.

All eight-step features use uniform weights $\alpha_k = 1/8$ in Equation (3). If Base+ACPC$_8$ improves on Base, multi-step information helps; if it also improves on Base+Control$_8$, the gain is attributable to the recorded actions rather than to merely rolling out eight steps.

Results. Base+ACPC$_8$ attains the lowest cross-validated MAE in all 12 task–run cells (Figure 4). For each training run we average the four task-level relative MAE reductions equally and report the mean $\pm$ sample standard deviation across the three training runs: the reduction is $55.9 \pm 4.7\%$ relative to Base and $51.3 \pm 3.5\%$ relative to Base+Control$_8$. These results concern prediction of the error drift; they do not compare the absolute forecast accuracy of one-step and eight-step world models. Appendix D reports the task-level values and selected controls (Tables 4 and 5). Eight-step ACPC carries information about the bounded error change that neither encoder shift, one-step ACPC, nor any same-horizon destroyed-action control provides.

4.5 ACPC and CEM Selection Regret

A prediction change matters to planning only if it affects the plan that CEM selects. The fixed-pool bound motivates the quantity below, but adaptive CEM can generate different later pools once the two elite sets diverge. We therefore evaluate full paired CEM runs rather than treating the fixed-pool condition as a run-level certificate. Our downstream quantity is *clean-reference selection regret*. Let $\mathbf{a}$ and $\tilde{\mathbf{a}}$ be the final plans selected from the clean and perturbed histories, respectively. We measure $(C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a}))_+$: the extra clean-history model cost incurred by selecting the perturbed-history plan. We refer to this single-decision quantity as *CEM selection regret*. It uses the model's squared latent goal cost; it is neither cumulative regret nor simulator return.

The paired CEM branches follow Appendix H: same initial proposal, common random numbers, each branch updating from its own elites. For every candidate, we compute one- and five-step ACPC along its action sequence

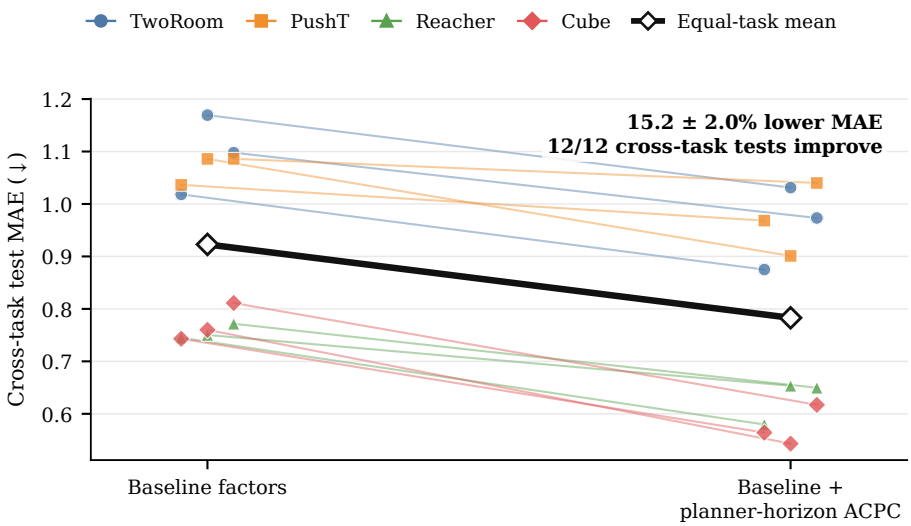

Figure 5: Adding planner-horizon ACPC improves prediction of adaptive-CEM selection regret on tasks excluded from regression fitting. Each colored line connects the baseline and expanded-model errors for one test task in one training run; the thick black line is the equal-task average across the three runs. The added feature is candidate-specific ACPC q90 at the planner's five-step prediction horizon. The baseline already contains severity, training condition, one-step ACPC q90, and the clean best–second-best cost gap. MAE is computed on $\log(1 + \text{selection regret})$.

and take the pool q90; five steps match the planner's prediction horizon. We compare two ridge regressions. The baseline uses perturbation severity, training condition, candidate-level one-step ACPC q90, and the clean gap between the best and second-best candidates. The expanded model adds candidate-level five-step ACPC q90. Within each training run, both regressions are fitted on three tasks and tested on the remaining task, rotating which task is excluded from fitting. We evaluate MAE on $\log(1 + \text{selection regret})$; Appendix H gives the CEM budget and sample counts.

Adding planner-horizon ACPC lowers cross-task test MAE by $15.2 \pm 2.0\%$ (mean $\pm$ sample standard deviation across training runs). The error decreases in all 12 task–run test cases (Figure 5). Since the two regressions differ only by the planner-horizon feature, this gain shows that the planner-horizon rollout carries additional cross-task predictive information. The comparison does not separate action conditioning from horizon length; it tests what the five-step ACPC feature adds. Candidate-specific ACPC therefore helps predict the clean-model cost of a perturbation-induced CEM selection change across tasks. It does not show that ACPC changes the selected action or improves environment performance.

4.6 Checkpoint Screening across Tasks

Because both the augmentation sweep and the threshold grid are discrete, we test for a useful shared threshold range rather than an exact universal boundary. A decision is correct when it matches the success-rate criterion of Section 4.1. We also count the grid steps between the first accepted checkpoint and the first checkpoint that meets that criterion.

Across the 14 source/test splits, 13 choose $(t_{\mathrm{IR}}, t_{\mathrm{SR}}) = (0.3, 0.95)$; only the Reacher-only split chooses $(0.1, 0.95)$. With two or three source tasks, the first accepted checkpoint is within 0.5 grid steps of recovery on average and never differs by more than one step (balanced accuracy 0.900, precision/recall 0.913/0.953). Single-source selection is less reliable: balanced accuracy averages 0.855 and ranges from 0.723 to 0.913. Reacher alone chooses the tighter IR threshold and delays acceptance on the other tasks by as many as five levels (Appendix E).

The predominant rule accepts a checkpoint only when relative IR is at most 0.3 and SR is at least 0.95. IR limits same-history sensitivity, while SR requires the tested state-coordinate distinctions to remain separated. Across the sweep, both measures improve over augmentation levels associated with recovery (Figure 3). Because $t_{\mathrm{IR}} = 0.3$ is the largest tested value, the upper edge of the useful range remains unresolved.

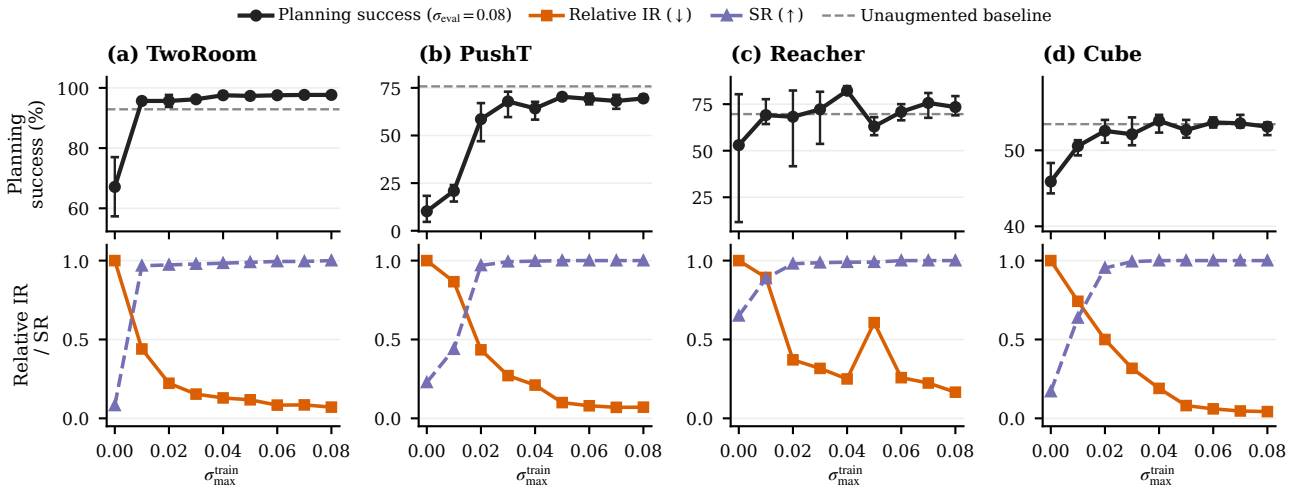

Figure 6: PLDM Gaussian-noise training sweep over three independent training runs. Curves show run means, and success-rate error bars span the run range. The dashed gray line marks clean success of the unaugmented checkpoint, and the leftmost black point is its success at evaluation noise $\sigma = 0.08$. Relative IR equals one at that checkpoint. Across tasks, stronger augmentation generally lowers relative IR and raises SR.

4.7 Diagnostic Behavior on PLDM

We apply the same ACPC, IR, and SR definitions to PLDM, which uses a different architecture and training recipe. The experiment repeats the nine-checkpoint Gaussian-noise training sweep, paired histories, and eight-step rollouts.

Across the PLDM sweeps, higher planning success under visual perturbation is generally accompanied by lower relative IR and higher SR, although the strength of this relationship varies across tasks.

4.8 Checkpoint Comparison under Blur and Resize

To test whether the diagnostics remain informative beyond Gaussian noise, we evaluate one blur severity ($k = 15$) and one resize severity (scale 0.25). For each task, training run, and visual shift, we compare the unaugmented checkpoint with the checkpoint trained at $\sigma_{\max} = 0.08$, giving $4 \times 3 \times 2 = 24$ pairs. We recompute IR and SR under the evaluated shift and use ΔS to compare the joint diagnostic scores of the two checkpoints. A positive ΔS favors the augmented checkpoint. The thresholds selected under Gaussian noise remain fixed, so this experiment evaluates relative checkpoint ordering rather than an absolute pass decision.

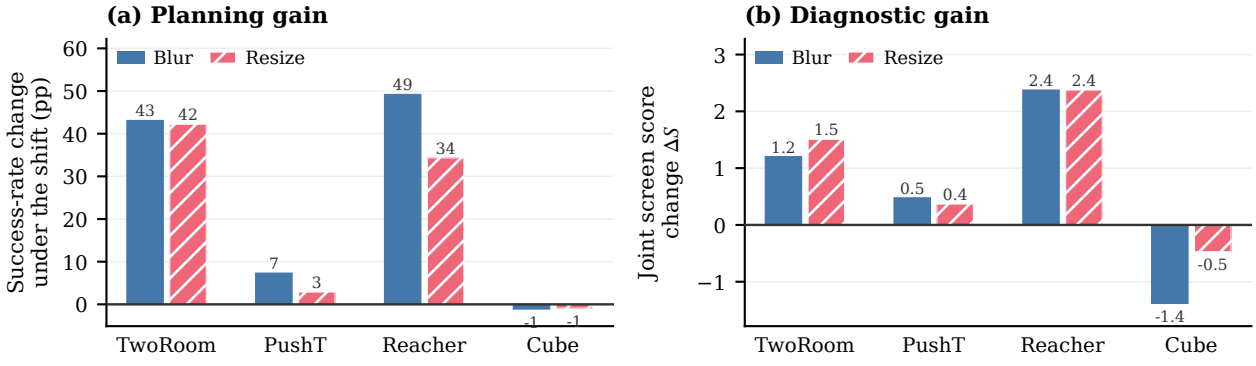

Figure 7: Relative checkpoint comparison under blur and resize. Bars average the three training runs for each task and shift; per-pair values are in Table 8. Panel (a) shows the change in planning success from the unaugmented to the $\sigma_{\max} = 0.08$ checkpoint. Panel (b) shows the change in the IR–SR score of Equation (12); positive favors the augmented checkpoint but is not an absolute pass decision.

Across the 24 pairs, the sign of ΔS agrees with the predefined pairwise criterion in 22 cases (balanced

accuracy 0.889), and its magnitude tracks the change in planning success (Spearman $\rho = 0.835$). The two nominal disagreements are boundary cases: both ΔS and planning success increase, but the observed success gain is four percentage points, just below the prespecified five-point cutoff. They therefore arise from thresholding a finite-sample estimate rather than from opposing diagnostic and behavioral trends. We interpret these results as evidence for relative checkpoint ordering, not as a binary robustness decision.

5 Discussion and limitations

What is supported. For individual clean–perturbed pairs, multi-step ACPC captures changes in prediction error more accurately than encoder distance, one-step ACPC, or controls that remove the recorded action sequence. At the planning horizon, ACPC also captures changes in the cost of the trajectory selected by CEM more accurately than one-step ACPC or the clean CEM margin. Together, these results support the central claim behind our analysis: rolling paired observations forward under the same action sequence reveals downstream effects that encoder-only and one-step comparisons can miss.

Across checkpoints, planning success under Gaussian noise generally improves as relative IR decreases and SR increases. Thresholds chosen using a subset of tasks also identify recovery on tasks that were not used to choose them. Under blur and resize, larger increases in the joint IR–SR score generally correspond to larger gains in planning success. The fixed five-point criterion produces only two boundary cases, each with a four-point success gain.

Why IR and SR are used together. IR can be low even when the representation collapses, because collapse brings all predicted states closer together. SR prevents such a model from being judged favorably by checking that states with different labels remain distinguishable after rollout. SR alone is also insufficient because it does not measure whether clean and perturbed observations remain close. IR therefore measures sensitivity to visual perturbations, while SR measures whether relevant state distinctions are preserved. The two measures should be considered together.

Scope and limitations. ACPC diagnoses how visual perturbations affect predicted rollouts and planning costs; it does not modify CEM. Our planner experiment therefore measures whether ACPC reflects the extra predicted cost of a perturbation-induced plan change, rather than whether ACPC-guided planning improves task success. The adaptive-CEM guarantee applies only while its bound verifies that both runs select the same elite candidates. Our checkpoint experiments cover Gaussian-noise augmentation and one severity each of blur and resize. The chosen IR threshold is also at the edge of the tested range, so larger values remain to be evaluated.

Future work. Future work can test whether using ACPC during planning improves task success. Broader evaluation should include multiple perturbation severities, more varied state pairs, additional robustness-training methods, and other JEPA world models.

6 Conclusion

ACPC measures how much a clean observation and its visually perturbed counterpart diverge after both are rolled forward under the same action sequence. We show that this divergence bounds changes in multi-step prediction error and predicted planning costs. IR summarizes perturbation sensitivity across a checkpoint, while SR checks whether different states remain distinguishable after rollout. Experiments show that multi-step ACPC captures changes in prediction error and CEM selection cost that encoder-only and one-step comparisons miss. Across checkpoints, lower IR and higher SR generally align with better planning performance under visual perturbations, and thresholds selected on some tasks identify recovery on the remaining tasks. Similar diagnostic trends appear in PLDM, while the blur and resize results extend the evidence beyond Gaussian noise.

References

[1] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[2] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[3] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022. Version 0.9.2.

[4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[6] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint-embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 21897–21937, 2025.

[7] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 91300–91336, 2024.

[8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[9] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[10] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[11] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[12] Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 43905–43941, 2025.

[13] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[14] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. arXiv preprint arXiv:2211.10831, 2022. Self-Supervised Learning: Theory and Practice Workshop at NeurIPS 2022.

[15] Zhaohan Daniel Guo, Bernardo Avila Pires, Bilal Piot, Jean-Bastien Grill, Florent Altché, Rémi Munos, and Mohammad Gheshlaghi Azar. Bootstrap latent-predictive representations for multitask reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3875–3886. PMLR, 2020.

[16] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[17] Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Avila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 33632–33656. PMLR, 2023.

[18] Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, Yunhao Tang, Clare Lyle, Mark Rowland, Nicolas Heess, Diana L. Borsa, Arthur Guez, and Will Dabney. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 181–189. PMLR, 2025.

[19] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model? arXiv preprint arXiv:2605.26379, 2026.

[20] Hafez Ghaemi, Eilif B. Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 32943–32973, 2025.

[21] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[22] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[23] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[24] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[25] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[26] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[27] Tung D. Nguyen, Rui Shu, Tuan Pham, Hung Bui, and Stefano Ermon. Temporal predictive coding for model-based planning in latent space. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8130–8139. PMLR, 2021.

[28] Fei Deng, Ingook Jang, and Sungjin Ahn. DreamerPro: Reconstruction-free model-based reinforcement learning with prototypical representations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 4956–4975. PMLR, 2022.

[29] Xiang Fu, Ge Yang, Pulkit Agrawal, and Tommi Jaakkola. Learning task informed abstractions. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3480–3491. PMLR, 2021.

[30] Tongzhou Wang, Simon S. Du, Antonio Torralba, Phillip Isola, Amy Zhang, and Yuandong Tian. Denoised MDPs: Learning world models better than the world itself. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 22591–22612. PMLR, 2022.

[31] Minting Pan, Xiangming Zhu, Yunbo Wang, and Xiaokang Yang. Iso-Dream: Isolating and leveraging noncontrollable visual dynamics in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 23178–23191, 2022.

[32] Ruixiang Sun, Hongyu Zang, Xin Li, and Riashat Islam. Learning latent dynamic robust representations for world models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47234–47260. PMLR, 2024.

[33] Yucen Wang, Shenghua Wan, Le Gan, Shuai Feng, and De-Chuan Zhan. AD3: Implicit action is the key for world models to distinguish the diverse visual distractors. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51546–51568. PMLR, 2024.

[34] Miles Hutson, Isaac Kauvar, and Nick Haber. Policy-shaped prediction: Avoiding distractions in model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 13124–13148, 2024.

[35] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[36] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[37] Christopher Grimm, André Barreto, Satinder P. Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5541–5552, 2020.

[38] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[39] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction. arXiv preprint arXiv:2603.07799, 2026.

[40] Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pages 761–770. PMLR, 2020.

[41] Ran Wei, Nathan Lambert, Anthony D. McDonald, Alfredo Garcia, and Roberto Calandra. A unified view on solving objective mismatch in model-based reinforcement learning. *Transactions on Machine Learning Research*, 2024.

[42] Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. *arXiv preprint arXiv:2606.15032*, 2026.

[43] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models. arXiv preprint arXiv:2606.09028, 2026.

[44] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding. arXiv preprint arXiv:2606.31232, 2026.

[45] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models. arXiv preprint arXiv:2607.02403, 2026.

[46] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? Diagnosing dynamic consistency in world action models. arXiv preprint arXiv:2605.07514, 2026.

[47] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions. arXiv preprint arXiv:2605.24578, 2026.

[48] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure. arXiv preprint arXiv:2607.05966, 2026.

[49] Jewon Yeom, Hanseul Kim, Jeongjae Park, Sungmok Jung, Jaejin Lee, and Taesup Kim. What makes video world model latents action-relevant: Prediction over reconstruction. *arXiv preprint arXiv:2606.07687*, 2026.

[50] Zhixiang Guo, Siyuan Liang, Shi Fu, Cheng Guo, András Balogh, Márk Jelasity, and Dacheng Tao. WMAttack: Automated attack search for adversarial evaluation of world-model agents. *arXiv preprint arXiv:2605.23220*, 2026.

[51] Junjian Zhang, Hao Tan, Ruonan Li, Dong Zhu, Aiping Li, and Zhaoquan Gu. ARB4WM: An adversarial robustness benchmark for world models in continuous control. *arXiv preprint arXiv:2606.16605*, 2026.

[52] Donna Vakalis. Operator-on-F complements value-equivalence: A planning-time diagnostic for latent world models. *arXiv preprint arXiv:2607.04464*, 2026.

[53] Hanzhe You, Yonggang Zhang, Maohao Ran, Zhiqin Yang, Zhenyuan Zhang, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. A control theory of predictability in latent world models. *arXiv preprint arXiv:2607.10362*, 2026.

[54] Björn Lütjens, Michael Everett, and Jonathan P. How. Certified adversarial robustness for deep reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 1328–1337. PMLR, 2020.

[55] Fan Wu, Linyi Li, Zijian Huang, Yevgeniy Vorobeychik, Ding Zhao, and Bo Li. CROP: Certifying robust policies for reinforcement learning through functional smoothing. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[56] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[57] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In Lise Getoor and Tobias Scheffer, editors, *Proceedings of the 28th International Conference on Machine Learning*, pages 833–840. Omnipress, 2011.

[58] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 18(3):1059–1076, 1989.

A Proofs and Fixed-Pool Analysis

Proof of Proposition 1. Let $u = \bar{G}_{\mathbf{a}}(E_\theta(h))$, $v = \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))$, and $y = Y^H_{\mathbf{a}}$. Both errors use the same target y in the weighted rollout space of Equation (2). The reverse triangle inequality gives

$$|e_{\tilde{h}} - e_h| = \big|\, \|v - y\|_2 - \|u - y\|_2 \,\big| \tag{13}$$

$$\leq \|(v - y) - (u - y)\|_2 = \|v - u\|_2 = \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{14}$$

The one-sided error increase satisfies the same bound because $(e_{\tilde{h}} - e_h)_+ \leq |e_{\tilde{h}} - e_h|$.

The following result uses the cost-change bounds for individual candidates to determine whether the selected candidate or elite set can change.

Proposition 3 (Fixed-pool selection certificates)**.** *In the setting of Proposition 2, let w be the unique winner for the clean input. If its cost advantage over every other candidate exceeds the combined cost-change bounds,*

$$\min_{j \neq w} \big(C_j - C_w - b_j - b_w\big) > 0, \tag{15}$$

then it remains the winner for the perturbed input. Similarly, let $\mathcal{E}$ be the clean top-k elite set. If

$$\min_{i\in\mathcal{E},\, j\notin\mathcal{E}} \big(C_j - C_i - b_j - b_i\big) > 0, \tag{16}$$

then the perturbed input produces the same elite set.

Corollary 1 (Conditional CEM stability)**.** *Consider clean and perturbed CEM runs that start from the same proposal and use the same random samples. Suppose that, at a given iteration, they evaluate corresponding candidate action sequences. If Equation* (16) *holds, they select the same elite candidates and fit the same proposal for the next iteration. If the condition holds at every iteration, the two runs remain aligned. Because the evaluated solver returns the final proposal mean as its action, the selected actions are also identical.*

Proof of Proposition 2. For a single candidate, omit the index. The difference between the two squared costs factors as

$$\begin{aligned} |\tilde{C} - C| &= \big|\|\tilde{x} - g\|_2^2 - \|x - g\|_2^2\big| && (17)\\ &= \big|\langle \tilde{x} - x, \tilde{x} + x - 2g\rangle\big| && (18)\\ &\le \|\tilde{x} - x\|_2 \, \|\tilde{x} + x - 2g\|_2 && (19)\\ &\le r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big) = b. && (20) \end{aligned}$$

The first inequality follows from Cauchy–Schwarz, and the second follows from the triangle inequality.

Let w and $\tilde{w}$ be the winners for the clean and perturbed inputs. Since $\tilde{w}$ minimizes the perturbed cost,

$$C_{\tilde{w}} \le \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \le \tilde{C}_w + b_{\tilde{w}} \le C_w + b_w + b_{\tilde{w}},$$

Since w minimizes the clean cost, $C_{\tilde{w}} - C_w \ge 0$. Together, these inequalities prove Equation (6).

For any clean winner w and competitor j,

$$\tilde{C}_j - \tilde{C}_w \ge C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \ge C_j - C_w - b_j - b_w.$$

Therefore, Equation (15) ensures that every competitor remains more costly than w. Applying the same argument to every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (16) preserves the elite set. The strict inequalities exclude ties and make the result independent of the tie-breaking rule.

Proof of Corollary 1. Index the clean and perturbed runs by $b \in \{\mathrm{c}, \mathrm{p}\}$ and the CEM iterations by t. Let ϕ_t^b denote the proposal mean and variance for run b. Given shared random samples ω_t, the deterministic sampling map T produces the candidate pool

$$\mathcal{A}_t^b = T(\phi_t^b, \omega_t).$$

Each run selects the indices $\mathcal{E}_t^b$ of its k lowest-cost candidates. A deterministic, permutation-invariant update U then fits the next proposal:

$$\phi_{t+1}^b = U(\{\mathbf{a}^j : j \in \mathcal{E}_t^b\}).$$

The evaluated solver satisfies this condition because it uses the unweighted mean and standard deviation of the elite candidates.

The two runs start from the same proposal, so $\phi_0^{\mathrm{c}} = \phi_0^{\mathrm{p}}$. If their proposals agree at iteration t, the shared samples generate the same candidate pool. When Equation (16) holds, the two runs also select the same elite candidates and therefore fit the same next proposal:

$$\phi_t^{\mathrm{c}} = \phi_t^{\mathrm{p}} \implies \mathcal{A}_t^{\mathrm{c}} = \mathcal{A}_t^{\mathrm{p}} \implies \mathcal{E}_t^{\mathrm{c}} = \mathcal{E}_t^{\mathrm{p}} \implies \phi_{t+1}^{\mathrm{c}} = \phi_{t+1}^{\mathrm{p}}.$$

Thus, if the condition holds at every iteration, the proposals and candidate pools remain identical. The evaluated solver returns the final proposal mean, so the selected actions are also identical.

If a solver instead returns the best candidate from the final pool, Equation (15) must also hold at the last iteration. If either certificate cannot be verified, the proof no longer guarantees that the subsequent elite sets, proposals, or candidate pools agree.

Relation to ACPC. The planning-cost bound uses the final-step displacement r_j, whereas ACPC combines displacements across all H rollout steps. From Equation (3),

$$\sqrt{\alpha_H}\, r_j \le \mathrm{ACPC}_H \qquad \text{when } \alpha_H > 0.$$

Thus, $r_j \le \mathrm{ACPC}_H/\sqrt{\alpha_H}$. In the planner experiments, we compute r_j directly instead of using this upper bound. This gives a tighter candidate-level measurement and does not require the observed future, but it requires rolling out each candidate from both the clean and perturbed histories.

Tightness of the bounds. Both bounds can hold with equality, so their right-hand sides cannot be reduced without additional assumptions. For the prediction-error bound, let u be a unit vector, set the common target to $Y = 0$, and let the two predicted rollouts be su and tu, where $s, t \ge 0$. Then

$$\big|\, \|tu\|_2 - \|su\|_2 \,\big| = \|tu - su\|_2,$$

so Equation (4) holds with equality. For the planning-cost bound, set $g = 0$, $x = su$, and $\tilde{x} = tu$. Then

$$|\tilde{C} - C| = |t - s|(t + s) = r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big),$$

so the candidate cost-change bound is also attained exactly.

B Local-Geometry Case-Study Definitions

This section defines the high-dimensional geometric quantities reported in the case study of Section 4.2. We use 128 logged PushT histories. For each history, we generate one clean view and 18 perturbed views, with six draws at each standard deviation in $\{0.01, 0.04, 0.08\}$. Each of the four qualitative t-SNE panels uses a separate deterministic projection, so coordinates are not compared across panels.

We evaluate two representation spaces: the encoder output, denoted by $\mathcal{V} = \mathrm{enc}$, and the representation after eight autoregressive rollout steps, denoted by $\mathcal{V} = \mathrm{roll}$. The rollout horizon is $H = 8$ (Section 3.1). For history i, let $v_{i,\mathcal{V}}^{(0)}$ be its clean representation and let $v_{i,\mathcal{V}}^{(m)}$, $m = 1, \dots, M$ with $M = 18$, be its perturbed representations. In the rollout space, the clean and perturbed views of the same history use the same recorded action sequence. We then define the sampled visual radius, the distance to the nearest other clean history, and their ratio:

$$\begin{aligned} r_{i,\mathcal{V}}^{\mathrm{vis}} &= \max_{1 \le m \le M} \left\| v_{i,\mathcal{V}}^{(m)} - v_{i,\mathcal{V}}^{(0)} \right\|_2, \\ d_{i,\mathcal{V}}^{\mathrm{NN}} &= \min_{j \ne i} \left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2, \qquad \rho_{i,\mathcal{V}}^{\mathrm{local}} = \frac{r_{i,\mathcal{V}}^{\mathrm{vis}}}{d_{i,\mathcal{V}}^{\mathrm{NN}}}. \end{aligned} \tag{21}$$

The figures abbreviate $\rho_{i,\mathcal{V}}^{\mathrm{local}}$ as r/NN. A value below one means that every sampled perturbation moves the representation by less than the distance from its clean anchor to any other clean anchor. A value of at least one means that at least one perturbation displacement is as large as the nearest-clean distance. This comparison uses only distance magnitudes: it does not show that a perturbed representation approaches or overlaps another history, and the nearest clean history need not differ across a task-relevant state boundary.

The r/NN ratio considers only the perturbation radius of anchor i. To account for the perturbation radii of both histories, we also compare their enclosing balls. The ball around anchor i is *fully disjoint* if it does not intersect the ball around any other anchor:

$$\left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2 > r_{i,\mathcal{V}}^{\mathrm{vis}} + r_{j,\mathcal{V}}^{\mathrm{vis}} \quad \text{for every } j \ne i. \tag{22}$$

Across the 128 anchors, we report three summaries: the median r/NN ratio, the fraction with $r_{i,\mathcal{V}}^{\mathrm{vis}} < d_{i,\mathcal{V}}^{\mathrm{NN}}$, and the fraction satisfying the fully disjoint condition in Equation (22). The second summary compares each perturbation radius with the distance to the nearest other clean anchor. The third also accounts for the perturbation radius around every other anchor. All three summaries are computed in the original high-dimensional representation; overlap between the qualitative t-SNE ellipses is not used.

These summaries are descriptive and depend on the sampled histories and perturbation draws. They compare the encoder output and the final rollout step but do not examine intermediate predictions. They also do not bound changes in prediction error or planning cost; those theoretical links are provided by pairwise ACPC in Section 3.1.

C Evaluation and Diagnostic Protocol

This appendix specifies the fixed evaluation and diagnostic protocol used in the main tables.

Evaluation grid. We evaluate LeWM on PushT, TwoRoom, Reacher, and Cube. Each checkpoint is evaluated with seeds 42, 43, and 44, using 100 episodes per seed. The primary evaluation adds Gaussian noise with standard deviation 0.08 to the history observations while keeping the goal image clean. For each task, the training grid contains one unaugmented checkpoint and eight checkpoints trained with full-sequence Gaussian-noise augmentation at $\sigma_{\max} \in \{0.01, 0.02, \ldots, 0.08\}$.

PLDM protocol. PLDM is evaluated on the same four tasks and nine-checkpoint training grid as LeWM, using the same evaluation seeds and number of logged histories. Its diagnostics use the same clean–perturbed pairing, eight-step rollout horizon, and SR definition. In the PLDM plots, raw IR is divided by the value of the unaugmented PLDM checkpoint from the same task and training run.

Success-rate criterion. For each task and training run, let P_{base} be the noisy-observation success rate of the unaugmented checkpoint, and let P_{best} be the highest noisy-observation success rate in the nine-checkpoint sweep. A checkpoint meets the success-rate criterion if its noisy-observation success rate is at least

$$P_{\text{base}} + 0.8(P_{\text{best}} - P_{\text{base}})$$

and its clean-observation success rate is no more than five percentage points below that of the unaugmented checkpoint. At the task level, we call an augmentation level recovered only when this criterion holds in at least two of the three training runs.

IR protocol. We compute IR separately for each task, training run, and checkpoint. To express rollout divergence relative to the normal motion in each history, we first compute a clean-motion scale. For anchor i, let $u^{\text{obs}}_{i,0}$ be the projected embedding of the final history observation, and let $u^{\text{obs}}_{i,1:H}$ be the projected embeddings of the next H observed clean frames. We define

$$s_i = Q_{0.50}\Big(\big\{\|u^{\text{obs}}_{i,k} - u^{\text{obs}}_{i,k-1}\|_2\big\}_{k=1}^{H}\Big). \tag{23}$$

Thus, s_i is the median displacement between consecutive clean observations, including the transition from the final history observation to the first future observation.

For each anchor, we generate multiple perturbation draws using the visual shift being evaluated. The Gaussian-noise experiments use $\sigma = 0.08$; the blur and resize experiments apply their corresponding shifts. For every draw, we compute eight-step ACPC with uniform weights $\alpha_k = 1/H$ and normalize it by $s_i + \varepsilon_{\text{norm}}$. We average the normalized values across draws for each anchor and then take q90 across anchors. The result is raw IR in Equation (9).

SR compares different-state distances with raw IR from the same checkpoint. For the sweep plots, raw IR is divided by the value of the corresponding unaugmented checkpoint, as defined in Equation (11). Only the LeWM cross-task analysis uses relative IR for threshold selection. We use $\varepsilon_{\text{norm}} = 10^{-8}$ in Equation (8) and $\varepsilon_{\text{score}} = 10^{-12}$ in Equation (12). Table 1 reports the results for the tested rollout horizons and summary quantiles.

Table 1: Horizon and quantile sensitivity of the IR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the IR of the checkpoint trained at the highest augmentation level ($\sigma_{\max} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

	$q = 0.90$ by horizon				$H = 8$ by quantile		
Task	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

Table 2: Endpoint-state labels used to construct SR pairs. The state is taken at the eighth observed future step, and its coordinates are standardized using full-dataset statistics. Counts report eligible and skipped histories among the fixed 100 anchors. These labels define operational partitions for the diagnostic; they are not semantic or action-relevance annotations.

Task	Logged state field (dim.)	Label construction $\ell(y)$	Eligible / skipped
TwoRoom	`pos_agent` (2)	Median split of the standardized agent x coordinate (`pos_agent[0:1]`).	61 / 39
PushT	`state` (7)	Three-bit label from median splits of standardized block x, block y, and block angle (`state[2:5]`).	98 / 2
Reacher	`observation` (6)	Three-bit label from median splits of the two standardized joint velocities and their Euclidean norm (`observation[4:6]`).	100 / 0
Cube	`observation` (28)	Three-bit label from median splits of the first two standardized coordinates and $\|r\|_2$, where $r = \bar{o}_{0:3} - \bar{o}_{25:28}$.	100 / 0

SR protocol. SR follows Equation (10). We first define a local neighborhood using the standardized logged endpoint states. Let $d_{0.35}$ be the 35th percentile of all pairwise distances between different endpoints—that is, the q35 of all off-diagonal Euclidean distances. This cutoff keeps the comparisons local while retaining an eligible neighbor for most anchors; Table 2 reports the resulting counts.

For each anchor i, we select the nearest history $j(i)$ that has a different endpoint-state label and lies within distance $d_{0.35}$. If no such history exists, the anchor is excluded from SR. We roll both histories forward under the anchor's recorded action sequence $\mathbf{a}_i$. This sequence was not generally executed by the neighboring history, but using it for both rollouts prevents action differences from affecting the comparison. Their normalized rollout distance is

$$D_i^{\text{diff}} = \frac{\left\| \bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)})) \right\|_2}{s_i + \varepsilon_{\text{norm}}}. \tag{24}$$

The pair counts as separated when D_i^{diff} is greater than raw q90 IR plus the fixed margin 0.10.

For pairing, the endpoint state is taken after the eight observed future steps and therefore records the state reached under that trajectory's own actions. The dataset preprocessor standardizes each state coordinate, and neighborhood distances use the full standardized state vector. Endpoint-state labels are constructed by median splits of the task-specific coordinates listed in Table 2, using the fixed batch of 100 endpoints generated with anchor seed 9101. Because the logged endpoints, labels, and neighborhoods are fixed before checkpoint evaluation, the selected pairs and eligible-anchor counts are identical across checkpoints and training runs.

Threshold grids. For the LeWM cross-task evaluation, we search

$$t_{\text{IR}} \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}, \qquad t_{\text{SR}} \in \{0.80, 0.85, 0.90, 0.95\}.$$

For each nonempty subset of tasks used for threshold selection, we evaluate every threshold pair. We first minimize the mean number of augmentation-grid steps between the first checkpoint accepted by the IR–SR screen and the first checkpoint that meets the success-rate criterion. Remaining ties are resolved by, in order, fewer early acceptances, fewer late acceptances, higher balanced accuracy, a smaller t_{IR}, and a larger t_{SR}.

The selected thresholds are then applied unchanged to the tasks that were not used for their selection. Success-rate labels from those tasks and all blur or resize results are excluded from threshold selection.

Reproducibility. The commands used to rebuild the paper figures and tables are documented in `paper1/scripts/README.md` and `tools/README_paper1.md`. The released machine-readable summaries are sufficient for the reader-facing aggregation and plotting steps. Recomputing checkpoint-level diagnostics additionally requires the released checkpoints and datasets. The summaries record the training seeds, evaluation seeds, and protocol hashes used for each result.

Table 3: Summary of the Gaussian-noise training sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 3 shows every level). "Best" is the level with the highest mean planning success at evaluation noise $\sigma = 0.08$; arrows give the change from the unaugmented checkpoint to that level. Relative IR is lower-is-better, and SR is higher-is-better. The last column lists levels meeting the success-rate criterion (Section 4.1).

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative IR	SR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

D Prediction-Error Scope and Controls

We checked Proposition 1 for every logged eight-step pair and every candidate-specific five-step pair across all tasks, training runs, and training conditions. No numerical violations occurred. This check confirms that the implementation follows the inequality. It is not an independent statistical success count or evidence of model quality, because the inequality holds for every correctly computed pair.

The prediction-error and planning experiments answer different questions. The prediction-error experiment measures how a visual perturbation changes rollout error relative to the observed future under the recorded actions. The planning experiment instead computes ACPC for the action candidates evaluated by CEM and relates it to selection regret. We therefore treat the planning analysis as separate evidence rather than infer it from the prediction-error result (Section 4.5).

The prediction-error analysis uses only the unaugmented checkpoints and Gaussian-noise levels $\sigma \in \{0.02, 0.05, 0.08\}$. Table 4 reports the relative reduction in cross-validated MAE for each training run. Table 5 reports the corresponding MAE values and the selected destroyed-action controls. Base+ACPC_8 achieves lower MAE than both comparison models on all four tasks in every training run.

Table 4: Relative reduction in out-of-group MAE from Base+ACPC$_8$ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.4). Base+Control$_8$ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC$_8$ is best.

Training run (seed)	vs. Base	vs. Base+Control$_8$	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean ± SD	55.9 ± 4.7%	51.3 ± 3.5%	12/12

Table 5: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.4. "Selected control" is the destroyed-action control chosen by the per-cell oracle; "paired wins" counts the folds (of 16) in which Base+ACPC$_8$ beats Base+Control$_8$.

Task	run (seed)	Base	Base +Control$_8$	Base +ACPC$_8$	selected control	paired wins
TwoRoom	3072	2.535	2.099	0.755	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	0.866	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	1.006	swapped actions	13/16
PushT	3072	2.068	2.068	1.762	shuffled actions	11/16
PushT	3073	1.969	2.008	1.315	zeroed actions	13/16
PushT	3074	1.909	1.833	1.439	zeroed actions	13/16
Reacher	3072	1.358	1.097	0.288	shuffled actions	16/16
Reacher	3073	1.399	0.793	0.247	shuffled actions	16/16
Reacher	3074	1.409	1.136	0.263	shuffled actions	16/16
Cube	3072	1.171	1.037	0.489	zeroed actions	14/16
Cube	3073	1.416	1.212	0.500	zeroed actions	14/16
Cube	3074	1.061	1.008	0.552	shuffled actions	14/16

E Cross-Task Threshold Partitions

With four tasks, there are 14 nonempty choices of tasks for threshold selection: four single-task choices, six two-task choices, and four three-task choices. The selected thresholds are applied to the remaining task or tasks. Each evaluation includes all three training runs and all nine checkpoints for every remaining task. Checkpoints are not counted as independent samples: metrics first average the training runs within each evaluation task and then weight the evaluation tasks equally.

When thresholds are selected using only Reacher, the procedure chooses $(t_{\mathrm{IR}}, t_{\mathrm{SR}}) = (0.1, 0.95)$ instead of the predominant $(0.3, 0.95)$. Both threshold pairs locate Reacher recovery within one augmentation-grid step, but $t_{\mathrm{IR}} = 0.1$ matches the Reacher onset exactly and therefore wins the tie-break. Recovered Reacher checkpoints have relative IR well below 0.1, whereas the earliest recovered checkpoints on the other tasks remain above 0.1. The Reacher-only threshold therefore accepts recovery on those tasks too late, by as many as five augmentation levels.

Table 6: Joint IR/SR checkpoint screening with thresholds chosen on other tasks. Thresholds are selected on the threshold-selection tasks and applied unchanged to all test tasks. Metrics first average training runs within each test task and then weight tasks equally. Recovery-onset mismatch is measured in training-augmentation grid levels (one level is 0.01 in $\sigma_{\max}$).

Selection tasks	Test tasks	t_{IR}	t_{SR}	BA	P / R	Onset mismatch mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

F Cross-Stressor Pairs and Boundary Cases

Table 7: IR–SR scores under blur and resize. For each task, thresholds are chosen on the other three Gaussian-noise tasks and then fixed; each pair compares the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart. Balanced accuracy (BA), precision, and recall compare the sign of ΔS with the predefined five-point success criterion (Section 4.8). "Boundary" counts the two pairs with a four-point success gain and a positive ΔS.

Visual shift	Pairs	BA	Precision / recall	Spearman	Boundary
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Both nominal disagreements lie at the boundary of the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the prespecified five-point cutoff, while their IR–SR scores also increase. Thus, the diagnostic and behavioral changes have the same direction; only the thresholded labels differ. The complete table reports these cases alongside the aggregate classification metrics.

Table 8: All 24 LeWM checkpoint pairs evaluated under blur and resize. $\mathrm{IR}_{\mathrm{rel}}$ and SR are measured for the checkpoint trained with Gaussian-noise augmentation ($\sigma_{\max} = 0.08$); ΔP and ΔS are its success-rate and IR–SR score changes relative to the unaugmented checkpoint. The criterion is met when $\Delta P \geq 5$ percentage points with at most a five-point clean loss. Daggers mark the two boundary cases with a four-point success gain, one point below the fixed cutoff, and a positive ΔS.

Task	Seed	Shift	$\mathrm{IR}_{\mathrm{rel}}$	SR	ΔP	ΔS	Criterion / ΔS sign
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	met / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	met / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	met / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	met / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	met / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	met / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	met / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	not met / positive†
PushT	3073	blur	0.846	0.918	7.3	0.515	met / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	met / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	not met / positive†
PushT	3074	resize	1.173	0.939	-19.7	-0.577	not met / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	met / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	met / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	met / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	met / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	met / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	met / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	not met / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	not met / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	not met / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	not met / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	not met / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	not met / nonpositive

G Local Sensitivity under Gaussian Noise

To relate ACPC under Gaussian noise to local model sensitivity, consider a small isotropic perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$. Let J_E and J_G be the Jacobians of the encoder and rollout map, evaluated along the clean input. A first-order expansion gives

$$\Delta G = J_G J_E \, \delta x + o(\|\delta x\|_2).$$

Writing $A = J_G J_E$ and using $\mathbb{E}[\delta x \delta x^\top] = \sigma^2 I$, we obtain

$$\mathbb{E}\big[\|A\delta x\|_2^2\big] = \mathrm{tr}\big(A\,\mathbb{E}[\delta x \delta x^\top] A^\top\big) = \sigma^2 \,\mathrm{tr}(AA^\top) = \sigma^2 \|A\|_F^2, \tag{25}$$

Therefore, for small σ,

$$\mathbb{E}[\|\Delta G\|_2^2] \approx \sigma^2 \|J_G J_E\|_F^2.$$

The squared Frobenius norm of the composed Jacobian thus measures how strongly the encoder–rollout path locally amplifies Gaussian observation noise. Jacobian Frobenius norms have previously been used to measure and regularize local representation sensitivity [57]. We estimate

$$\|J_G J_E\|_F^2 = \mathrm{tr}\big[(J_G J_E)^\top (J_G J_E)\big]$$

with Rademacher probes and the Hutchinson estimator [58]. Jacobian–vector products compute these estimates without constructing the full Jacobian. In every task, both the finite-difference and Jacobian-based estimates are lower for the noise-augmented checkpoint (Figure 8). This result is consistent with Gaussian-noise augmentation reducing ACPC by lowering the local sensitivity of the encoder–rollout path. The analysis applies only near the evaluated inputs and does not provide a global robustness guarantee.

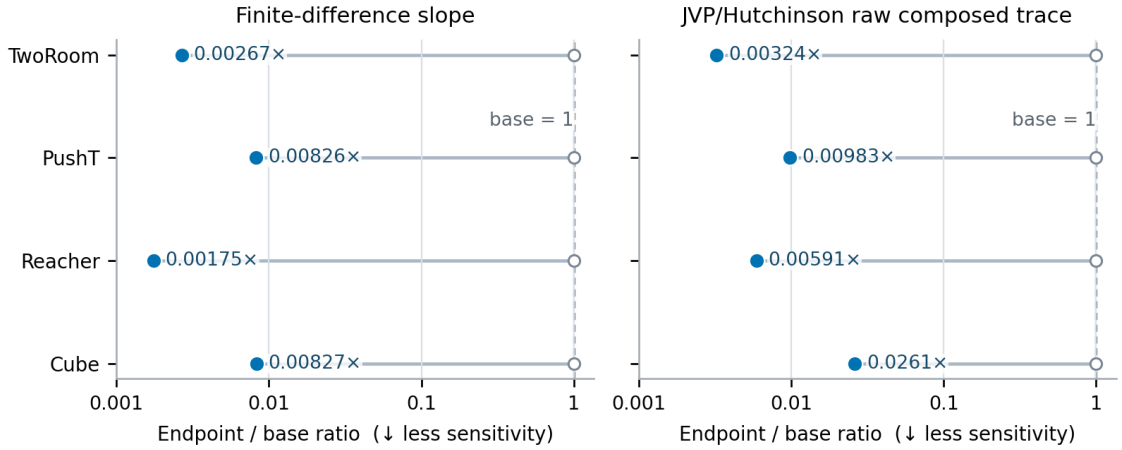

Figure 8: Ratio of local sensitivity for the Gaussian-noise-augmented checkpoint to that of the unaugmented checkpoint. We report a finite-difference estimate and a JVP-based Hutchinson estimate of the composed Jacobian's squared Frobenius norm. Both estimates are below one for every task, indicating lower local sensitivity after augmentation. These local measurements do not establish task performance or global robustness.

H Adaptive-CEM Selection-Regret Protocol

The selection-regret analysis does not use task-success labels. We evaluate the unaugmented and $\sigma_{\max} = 0.08$ checkpoints from every task and training run. For each checkpoint, we use 100 histories and apply perturbations at severities 0.02, 0.05, and 0.08. Each clean or perturbed CEM run uses 64 candidate action sequences, eight elite candidates, eight iterations, and a five-step prediction horizon. The paired runs start from the same proposal and use the same random samples, but each run updates its proposal using its own elite candidates.

The implementation averages the squared embedding error across the d embedding coordinates:

$$C_j = \frac{\|x_j - g\|_2^2}{d}.$$

This differs from the summed squared cost in Proposition 2 only by the positive factor $1/d$. The factor rescales every C_j and b_j equally, so the regret bound and selection certificates remain unchanged.

For each CEM run, we compute ACPC for every candidate under that candidate's action sequence. We summarize the candidate pool by the q90 ACPC at horizons one and five. Let $\mathbf{a}$ be the plan selected from the clean history and $\tilde{\mathbf{a}}$ the plan selected from its perturbed counterpart. The regression target is the extra predicted cost of the perturbed-history plan when both plans are evaluated from the clean history:

$$\big[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})\big]_+.$$

Within each training run, we fit ridge regressions on three tasks and evaluate them on the remaining task, rotating through all four choices. MAE is computed on $\log(1 + \text{selection regret})$ and averaged equally across the four evaluation tasks. The relative MAE reduction is computed separately for each training run before reporting the mean and sample standard deviation across the three runs. This experiment evaluates whether ACPC reflects the model-cost effect of a CEM selection change; it does not evaluate simulator return.